

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Thu 2 February 2017 - Part Theory

NAME (capital letters pls.):

MATRICOLA:

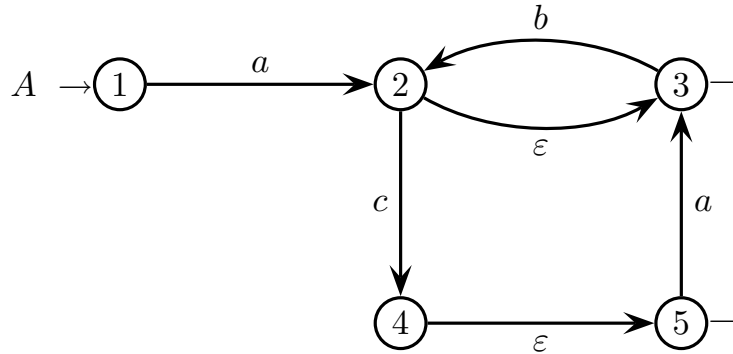
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the finite-state nondeterministic automaton A below, over the three-letter alphabet $\Sigma = \{ a, b, c \}$, with initial state 1 and final states 3, 5:



Answer the following questions:

- (a) By using the Berry-Sethi method applied to automaton A , find an equivalent deterministic automaton A' and minimize it if necessary.
 - (b) By using the node elimination method (Brzozowsky) applied to automaton A or automaton A' (at choice), find an equivalent regular expression R .
 - (c) Examine if the regular language $L(A)$ is local or not, and justify your answer.
 - (d) (optional) By cutting off the spontaneous transitions (ε -transitions) from automaton A , obtain an equivalent automaton A'' (not necessarily deterministic), and test it with all the strings of length ≤ 3 that belong to language $L(A)$.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider a three-letter alphabet $\Sigma = \{ a, b, c \}$, and the free language L below:

$$L = \{ x^R c y \mid x, y \in \{ a, b \}^+ \wedge (x = y \text{ or } x \text{ is a substring of } y) \}$$

Answer the following questions:

- (a) Write a *BNF* grammar G (no matter if ambiguous) that generates language L .
- (b) Draw a syntax tree for this sample valid string of language L :

$a b c b b a a$

- (c) (optional) Determine if grammar G is ambiguous and justify your answer: if it is ambiguous then provide a string with two or more syntax trees, else argue that it does not generate any ambiguous string.
-

2. Consider a description language that models the layout of a graphical terminal, consisting of a desktop populated by window objects, shortly *widgets*, and application programs, shortly *apps*. The widgets may contain other widgets or apps, at a finite yet unlimited depth. Such a language features the following structures:

- the desktop may be empty, or may contain widgets or apps (or both)
- a widget must first specify, in this order:
 - its left upper corner position (mandatory): **position** *x*, *y*
 - its (diagonal) size (optional): **diagonal** *d*
 - its aspect ratio (mandatory): **aspect** *a* or the unique keyword **square**
 - a header (mandatory): non-empty alphanumerical string
 where *x*, *y* and *d* are integers, and *a* is a fixed-point number (all are non-negative)
- then, a widget must contain one or more widgets or apps (or both), in any order
- an app is a program with these arguments:
 - an executable file (pathname) in the Linux form (mandatory): absolute from the root ‘/’, e.g., `exec /dir1/file`, or relative, e.g., `exec file`, `exec dir1/file2`; the current ‘.’ and parent ‘..’ directory symbols are allowed, e.g., `exec ./file`
 - an optional list of widgets to open and pop up on launch, and of functions, i.e., apps, to call on launch, in any order

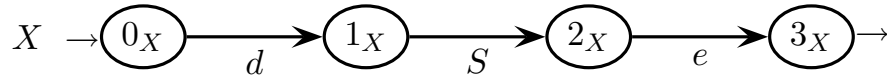
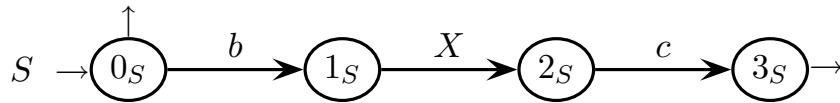
For any detail that may have been left unspecified, e.g., separators, etc, one can take inspiration from the sample description below:

```
desktop
  widget
    position 10, 20; diagonal 30; aspect 1.5; header editor;
    widget
      position 30, 40; square; header compiler;
      app ... end app
    end widget
    widget ... end widget;
    app exec ./file; end app
    app ... end app
    widget ... end widget
    ...
  end widget
  app
    exec /dir1/file;
    widget ... end widget
    app exec file; end app;
    ...
  end app
  ...
end desktop
```

Write a grammar *G*, *EBNF* and unambiguous, that models the sketched language.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ b, c, d, e \}$ and the nonterminal alphabet $V = \{ S, X \}$ (axiom S):



Answer the following questions:

- (a) Draw the complete pilot of grammar G , say if grammar G is of type $ELR(1)$ and shortly justify your answer. If grammar G is not $ELR(1)$ then highlight all the conflicts in the pilot.
- (b) Write the necessary guide sets on the arcs of the machine net and determine if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer. If you wish, you can use the figure above to add the call arcs and annotate the guide sets.
- (c) Sketch a simulation of the ELR analysis of the sample valid string below:

$b d b d e c e c$

and show the sequence of stack configurations and parser moves. In the stack it suffices to show the names I_0 , I_1 , etc, of the pilot states stored, alternated to the grammar symbols. Continue the table prepared on the next page.

- (d) (optional) Determine the computational complexity class of the function $h(n)$ that expresses the maximal height of the stack as a function of the length $n = |x|$ of the input string x .

4 Language Translation and Semantic Analysis 20%

1. Consider the arithmetic expressions with the operation of addition '+', variables and numbers schematized by letter a , and subexpressions enclosed in round brackets '()', and at least two addends at all levels. Here is a non-ambiguous *BNF* grammar G for their language (axiom S):

$$G \left\{ \begin{array}{l} 1: S \rightarrow E \\ 2: E \rightarrow T + E \\ 3: E \rightarrow T + T \\ 4: T \rightarrow a \\ 5: T \rightarrow (S) \end{array} \right.$$

We wish to translate such arithmetic expressions into a generalized postfix form with multi-operand addition, e.g., expression $a + a$ is translated into $\langle a a +$, expression $a + a + a$ into $\langle a a a +$, etc. Notice that the open angle bracket ' $\langle$ ' marks the head of the list of addends, which must have a length ≥ 2 in every (sub)expression. Here is a composite example with subexpressions:

$$a + ((b + b) + (c + c)) + a \quad \mapsto \quad \langle a \langle \langle b b + \langle c c + + a +$$

where various terminal letters are used as a visual help for the reader to understand.

Answer the following questions:

- (a) Test the source grammar G and draw the source syntax tree of the above composite example.
- (b) Use the proposed source grammar G and write a syntax translation scheme (or grammar) G_τ that computes the sketched translation. If you wish, you can complete the partial scheme already prepared on the next page.
- (c) Test your translation scheme (or grammar) G_τ and draw the destination syntax tree of the above composite example.
- (d) (optional) Say if your translation scheme (or grammar) G_τ is deterministic, and reasonably justify your answer.

syntax translation scheme G_τ to complete - question (b)

G_τ	{	<i>source grammar</i>	<i>destination grammar</i> (to be written)
		1: $S \rightarrow E$	
		2: $E \rightarrow T + E$	
		3: $E \rightarrow T + T$	
		4: $T \rightarrow a$	
		5: $T \rightarrow (S)$	

2. The phrases of a language consist of balanced sequences of nested round parentheses, e.g., $() \left(((()) ()) \right) ()$. The grammar below generates this language (axiom S):

$$\begin{cases} 1: S \rightarrow T \\ 2: T \rightarrow T T \\ 3: T \rightarrow (T) \\ 4: T \rightarrow () \end{cases}$$

The *nesting level* of a parenthesis pair is defined as usual, assuming that the value for such a level starts from 1 (for the outermost parentheses). Thus the expression:

$$() \left(((()) ()) \right) ()$$

contains three pairs at nesting level 1, two pairs at level 2 and one pair at level 3.

We intend to compute, in two suitable attributes of the root node of the syntax tree, the number of parenthesis pairs that have an odd nesting level (1, 3, etc) and the number of those that have an even nesting level (2, 4, etc). For instance, in the above example there are four pairs at an odd nesting level and two pairs at an even level.

The attributes to use are already assigned. See the table on the next page. Other attributes are unnecessary and should not be added.

Answer the following questions:

- Write an attribute grammar (in the table prepared on the next page), based on the above syntax, that defines the computation of the attributes *no* and *ne* in the tree root, as explained above. It is required to use the attributes illustrated in the table on the next page.
- Decorate the syntax tree of expression $() \left(((()) ()) \right) ()$ with the attribute values. Use the syntax tree prepared on the next page.
- (optional) Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers.

attributes assigned to be used				
<i>type</i>	<i>name</i>	<i>domain</i>	<i>(non)term.</i>	<i>meaning</i>
right	<i>nep</i>	integer	T	number of nested parenthesis pairs that enclose (are around) the current nonterminal
left	<i>no</i>	integer	T, S	number of parenthesis pairs at an odd nesting level that are contained in the subtree rooted at the current nonterminal
left	<i>ne</i>	integer	T, S	number of parenthesis pairs at an even nesting level that are contained in the subtree rooted at the current nonterminal

attribute grammar to write - question (a)

#	<i>syntax</i>	<i>semantics</i>
---	---------------	------------------

1: $S_0 \rightarrow T_1$

2: $T_0 \rightarrow T_1 T_2$

3: $T_0 \rightarrow (T_1)$

4: $T_0 \rightarrow ()$

syntax tree to decorate - question (b)

